

Concordia University

Team Project 3 – Animal Shelter Management System

CEWP MOD4-GRA Java Programming

1. Problem Definition

Scenario Chosen: Animal Shelter Management System

Problem Statement: An animal shelter needs a system to manage animals, handle adoptions, track medical history, and organize volunteer schedules. The shelter should be able to manage different types of animals (Dog, Cat, Bird), ensure adoption eligibility, record medical treatments, and allow volunteers to assist with daily operations. The system must support searching, sorting, and exception handling for invalid adoption scenarios.

Project Objectives (Aligned with EFREI Course Modules):

- **Modules 1–2:** Use loops, conditionals, and collections (ArrayList) to manage products and orders.
- **Modules 3–4:** Implement OOP principles: abstract classes, inheritance, polymorphism, encapsulation, and interfaces.
- **Modules 5–6:** Implement searching and sorting algorithms (linear/binary search, bubble/insertion/selection sort) for products.
- **Module 7:** Use recursion for price calculations or browsing categories.
- **Module 8:** Exception handling for invalid inputs or failed payments.
- **Module 9:** Compose classes like ShoppingCart and OrderHistory to manage collections of objects.
- **Module 10:** Implement a menu-driven console interface for browsing, adding to cart, paying, and reviewing.

2. Conceptual Design

2.1 Key Entities and Relationships

Entity	Type	Description	Relationships
Animal	Abstract Class	Common attributes: id, name, age, arrivalDate, adopted	Parent class
Dog	Class	Attributes: breed, isTrained	Inherits Animal
Cat	Class	Attributes: indoorOnly, color	Inherits Animal

Bird	Class	Attributes: wingSpan, species	Inherits Animal
Adoptable	Interface	Methods: adopt(), returnToShelter()	Implemented by Animal subclasses
MedicalRecord	Class	Vaccinations, treatments, notes	Composed within Animal
Shelter	Class	Manages animals and volunteers using ArrayList	Composition
AdoptionForm	Class	Adopter info, date, approval status	Linked to Animal
Volunteer	Class	Name, schedule, assigned tasks	Managed by Shelter
ShelterException	Custom Exception	Invalid adoption or animal state	Thrown during adoption

2.2 Features to Implement

Core Features:

- Add, remove, and display animals in the shelter.
- Adopt and return animals using the Adoptable interface.
- Track medical history for each animal.
- Manage volunteers, schedules, and assigned tasks.
- Display available animals using loops and conditionals.

Enhancement Features:

- **Searching:** Search animals by name or type.
- **Sorting:** Sort animals by age or arrival date.
- **Exception Handling:** Prevent adopting already-adopted animals.
- **Loops & Conditions:** Validate adoption eligibility.
- **Modularization:** User-defined methods for searching, sorting, and updates.

3. Class Design

3.1 UML Class Diagram

(To be created using draw.io, Lucidchart, or Figma)

Key elements to include:

- Abstract class **Animal**
- Subclasses: **Dog, Cat, Bird**
- Interface **Adoptable**
- **Shelter** class (composition with Animal and Volunteer)
- **MedicalRecord** and **AdoptionForm**
- Custom exception **ShelterException**

3.2 Inheritance & Polymorphism Examples

- **Inheritance:** **Dog, Cat, and Bird** inherit shared attributes and methods from **Animal**.
- **Polymorphism:** The shelter treats all animals as **Animal** objects while executing different **adopt()** behaviors depending on the animal type.
- **Interface Usage:** **Adoptable** ensures all adoptable animals follow the same adoption contract.

4. Sequence Diagram

Example: Borrowing a Book

1. User views available animals.
2. User selects an animal by ID.
3. System checks adoption eligibility.
4. Medical record is reviewed.
5. AdoptionForm is completed.
6. **adopt()** method is executed.
7. Animal status updated.
8. Confirmation displayed or exception thrown.

5. Implementation Plan

5.1 Package Structure

animal-shelter/

```
├─ model/
|   ├─ Animal.java
|   ├─ Dog.java
|   ├─ Cat.java
|   ├─ Bird.java
|   ├─ MedicalRecord.java
|   ├─ AdoptionForm.java
|   └─ Volunteer.java
├─ interfaces/
|   └─ Adoptable.java
├─ service/
|   ├─ Shelter.java
|   ├─ SearchUtils.java
|   └─ SortUtils.java
├─ exceptions/
|   └─ ShelterException.java
├─ ui/
|   └─ MainMenu.java
├─ data/
|   ├─ animals_data.txt
|   ├─ volunteers_data.txt
|   └─ adoptions_data.txt
```

5.2 Development Tasks

Member	Task
Member 1	Animal hierarchy and Adoptable interface
Member 2	Shelter class and animal management
Member 3	MedicalRecord and AdoptionForm
Member 4	Searching, sorting, and exception handling
All	Menu system, testing, documentation

6. Testing Plan

6.1 Test Scenarios

- Add animals → displayed correctly.
- Adopt animal → status updated.
- Adopt already adopted animal → exception thrown.
- Search by name/type → correct results.
- Sort by age or arrival date → correct order.
- Volunteer assignment → schedule updated.

6.2 Edge Cases

- Attempt to adopt a non-existent animal.
- Return an animal not adopted.
- Empty shelter display.
- Invalid volunteer schedule.

7. Reflection Questions

7.1 Adaptable Interface:

Using the Adoptable interface allows the shelter system to handle the adoption process uniformly for all animal types without depending on their specific implementations. This makes the system flexible and easy to extend in the future, for example when adding new animal types such as Rabbit or Reptile, without modifying existing shelter logic.

7.2 Polymorphism:

Polymorphism allows the shelter to manage different animals (Dog, Cat, Bird) using a common Animal reference. Each animal can implement its own adoption rules while the shelter processes them in a consistent way. This reduces code duplication and improves maintainability.

7.3 Composition vs Inheritance:

The Shelter class uses composition by containing Animal, MedicalRecord, and Volunteer objects rather than inheriting from them. This design is more appropriate because a shelter is not a type of animal or person—it is an entity that manages them. Composition provides greater flexibility and allows the system to scale as new services or roles are added.

8. References

- Oracle Java Documentation: <https://docs.oracle.com/javase/>
- Baeldung Java Tutorials: <https://www.baeldung.com/java>
- freeCodeCamp Java Tutorials: <https://www.freecodecamp.org/>
- UML diagram tools: draw.io, Lucidchart, Figma